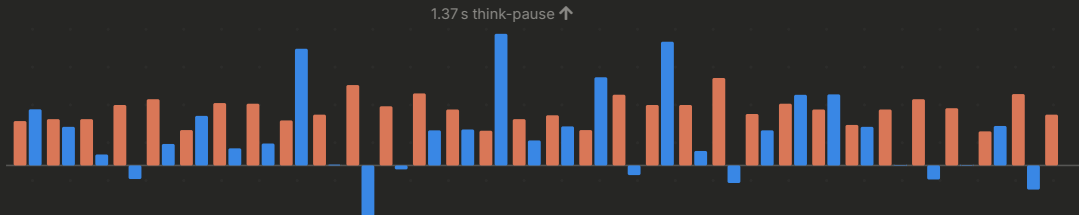


KEYSTROKE DYNAMICS · BEHAVIOURAL BIOMETRICS

KeyLoggd

Who's typing? Identifying people by the **rhythm** of their keystrokes

Team Ahis · September 29, 2026



■ dwell: key held ■ flight: gap to the next key (below the line = rollover) · a real 32-keystroke sample from our dataset

Today

01

Why keystrokes?

🕒 about 3 minutes

What is wrong with the logins we have, where typing rhythm stands out, what KeyLoggd is, how we turned enrollment and identification into a game, and where it falls short.

02

Under the hood

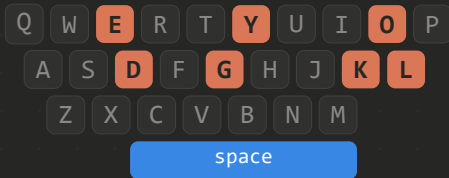
⚙️ every technical detail

Capture → signals → spectrum → 26 robust features → templates → distances → fused, thresholded verdict. Plus evaluation, results and how the app is engineered.

01

Why keystrokes?

The problem with logins, where typing rhythm stands out, what we built, how we gamified it, and its limits.



Every login factor has a weak spot



Something you *know*

password · PIN ·
pattern

- ✗ phished, guessed, leaked and *reused* across sites
- ✗ shoulder-surfed or simply shared
- ✗ forgotten → weak reset flows



Something you *have*

phone OTP · token ·
card

- ✗ SIM-swap, lost or stolen device
- ✗ OTPs can be phished in real time
- ✗ friction: fetch the phone, copy the code



Something you *are*

fingerprint · face · iris

- ✗ needs a dedicated sensor
- ✗ cannot be revoked once leaked
- ✗ spoofable with photos or moulds

THIS PROJECT



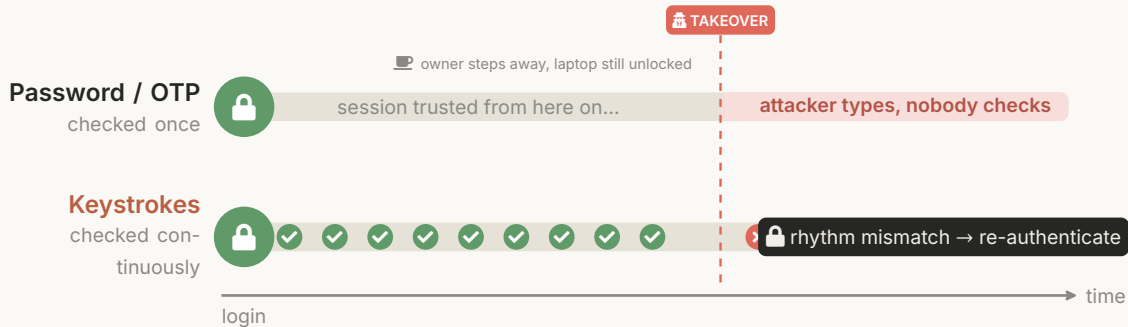
Something you *do*

how you type:
keystroke dynamics

- ✓ nothing to remember or carry
- ✓ any keyboard is the sensor
- ✓ happens every time you type, so it can keep checking

...and the first three check you **once**, at the door.

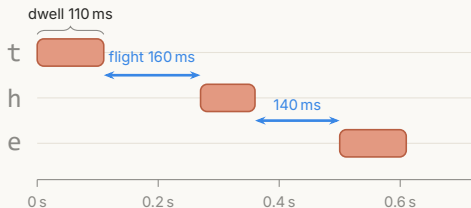
The one-time gate problem



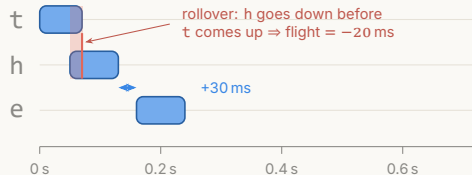
💡 Your typing never stops being *you*. Keystroke dynamics can turn authentication from a **door check** into a **continuous signal**, at no extra hardware cost.

Same word, different hands

Alice (deliberate)



Bob (fast, rolls keys)



Same letters, different signal

Hand size, finger habits, training and keyboard shape the timing: invisible, and hard to fake by watching.

From our real captures

arghya **never** rolls over (0 %); ahnaf does it on **36 %** of keys. Median gap: 2 ms vs 103 ms.

Where keystroke dynamics stands out

CRITERION	 Password	 OTP / 2FA	 Biometric	THIS PROJECT  Keystrokes
No extra hardware	✓	✗	✗	✓
Nothing to remember or carry	✗	✗	✓	✓
Can't be copied just by watching	✗	—	—	✓
Keeps checking <i>after</i> login	✗	✗	—	✓
Zero extra effort for the user	✗	✗	—	✓
Revocable if the template leaks	✓	✓	✗	—
Accuracy today	✓	✓	✓	—
Overall (strong 1, partial ½)	 3 / 7	 2.5 / 7	 3.5 / 7	 6 / 7

✓ strong — partial ✗ weak

Sweet spot: a free, invisible, *continuous* second signal.

KeyLoggd at a glance

“Type for thirty seconds. We’ll tell you who you are.”



Enroll

Take a Monkeytype-style typing test. Every keydown / keyup is timestamped; each run becomes 4–5 samples saved to `data/synthetic/<you>.json`.



Identify

Type again. The run is matched against every enrolled template and fused into one verdict: a name, or UN-RECOGNIZED.



Explain

Step through the pipeline on your own keystrokes: signal, FFT, features, z-scores, distances.

26

features per sample

32

keystrokes per sample

6

real typists · 48 samples

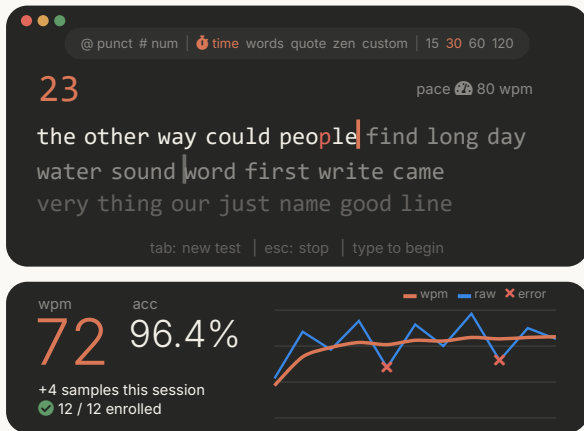
0

ML libraries, plain numpy

100%

benchmark accuracy, 5 samples fused

Gamified enrollment: the test is the sensor

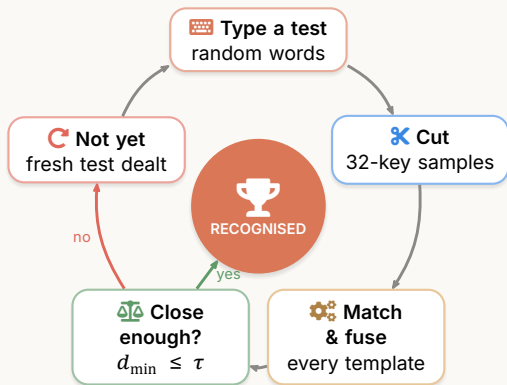


- **Monkeytype-style test:** *time* (15–120 s), *words* (10–100), *quote*, *zen*, *custom*; punctuation & numbers.
- **Live feedback:** per-character colour, a gliding caret, a ghost **pace caret** (40–300 wpm) to race.
- **Score screen:** net wpm, accuracy, per-second raw-wpm chart with error marks.
- **Visible progress:** "+4 samples added"; a milestone at 12 samples.

💡 Why a game?

Chasing a score, people type in their *natural* rhythm. One 30 s test yields **4 to 5 samples**.

Gamified identification: type until recognised



The name is then "typed out" letter by letter: a keystroke project answering in keystrokes.

Game mechanic

What it does for the biometric

Timer & wpm score

keeps people at their natural, fluent pace

Random word deals

no memorised phrase $\Rightarrow$ genuine free-text rhythm

Pace caret

a steady target, less hesitation noise

Instant retry loop

every miss adds samples to the next fused verdict

Verdict reveal

a pay-off moment; the *margin* shows confidence

Explain mode

curiosity $\rightarrow$ users see *why* they were recognised

Limitations, honestly

Data

- ✗ Only **6 real typists**, 48 samples, **one sitting** each.
- ✗ Real-data EER **19.0 %**, vs 5.9 % on the benchmark.
- ✗ New users shift everyone's normalisation.

Humans drift

- ✗ Fatigue, mood, injury, new keyboards: rhythm drifts.
- ✗ Key-class features assume English with spaces.
- ✗ Needs ≥ 12 samples (≈ 3 tests) per template.

Security

- ✗ No liveness check: replays or trained mimics pass.
- ✗ Sensitive keystroke logs are stored as plain JSON.
- ✗ Impostor accepts: ≈ 7 % on the benchmark, 22 % on real data.

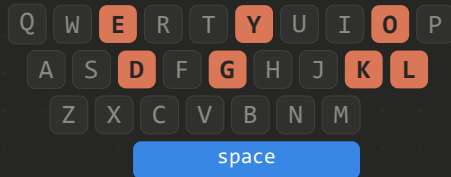
Engineering

- ✗ Tk timing: ≈ 5 ms bias, ≈ 3 ms animation jitter.
- ✗ Identifies typists; doesn't guard a real login yet.
- ✗ A desktop app, not a background OS-wide service.

02

Under the hood

From a key press to a verdict: capture, signals, spectrum, features, templates, decisions, and how we measured it.



Architecture: two halves that never mix

**keyloggd.ui**

Tkinter front end · standard library only · owns no maths

app

3 views, worker thread

typing_test

Monkeytype test

explain

6-phase walk-through

charts

canvas plots

splash

warm start

anim

60 fps tweens

theme

palette, fonts

**keyloggd.pipeline**

signal processing · numpy only · no Tk import

samples

32-key windows

signal_construction

dwell / flight

timing_features

26-D vector

fft_features

spectrum

denoise

low-pass

classifier

templates, EER

identify_sample

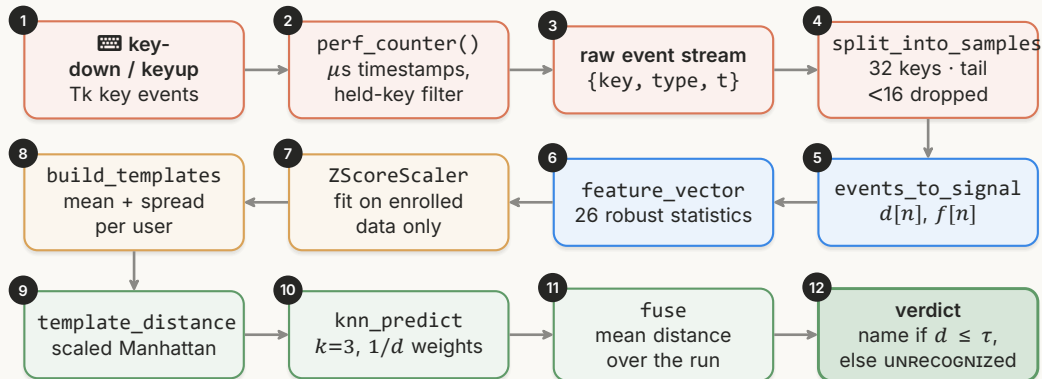
fuse, verdict

**tools/** synthetic_data_generator · benchmark_inference

simulated typists with rollover & pauses; the cross-phrase benchmark

data/synthetic/
one JSON file per typist

End-to-end data flow



capture

signal & features

model

decision

every box is a real function

Step 1 · Capture: timestamping every key

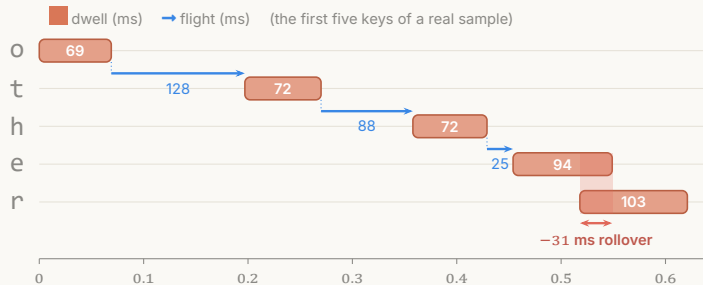
- Tk `<KeyPress>` / `<KeyRelease>` are routed into the typing test.
- `t = time.perf_counter()` – session start, rounded to $1\mu\text{s}$ (6 decimals).
- A **held-key set** drops OS auto-repeat: a second keydown with no keyup between would pair the wrong two timestamps.
- Only printable keys and space are recorded. **Backspace** edits the screen, not the signal; **Tab** deals a new test, **Esc** stops.
- The first keystroke *settles* any running animation, so it can't delay a later timestamp.

 data/synthetic/swayam.json

```
{ "user_id": "swayam",  
  "phrase": "free text (typing test)",  
  "samples": [  
    { "events": [  
      { "key": "o", "type": "down", "t": 0.0},  
      { "key": "o", "type": "up", "t": 0.069164},  
      { "key": "t", "type": "down", "t": 0.197467},  
      { "key": "t", "type": "up", "t": 0.269857},  
      ... ]},  
    { "phrase": "other me yopur ur ferom ...",  
      "captured_at": "2026-09-12T23:26:00" },  
    ... ] }
```

One schema everywhere: the synthetic generator, the typing test and the phrase widget all write it.

Step 2 · Dwell, flight and rollover



Two discrete signals

$$d[n] = t_n^{\uparrow} - t_n^{\downarrow}$$

$$f[n] = t_{n+1}^{\downarrow} - t_n^{\uparrow}$$

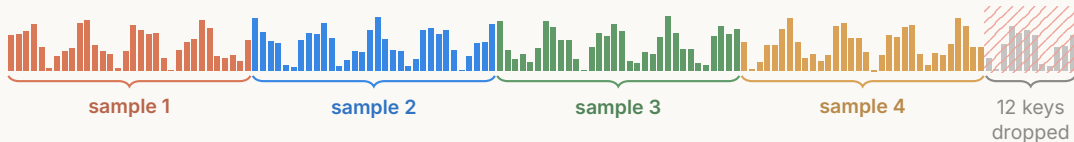
N keystrokes $\Rightarrow N$ dwells, $N-1$ flights.

$f[n] < 0 \Leftrightarrow$ **rollover**: the next key went down first.

Signal view. The true process is an impulse train $x(t) = \sum_i \delta(t - t_i)$ at the key-down times. $d[n]$ and $f[n]$ sample it *at the events* rather than at fixed time steps, and everything downstream treats them as discrete-time signals.

Step 3 · Slicing one long run into samples

one 30-second typing test $\approx$ 140 keystrokes $\rightarrow$ 4 samples



Pairing

Each *down* is matched to the next *up* of the same key, so overlapping keys (rollover) pair correctly. A down with no up is dropped, never guessed.

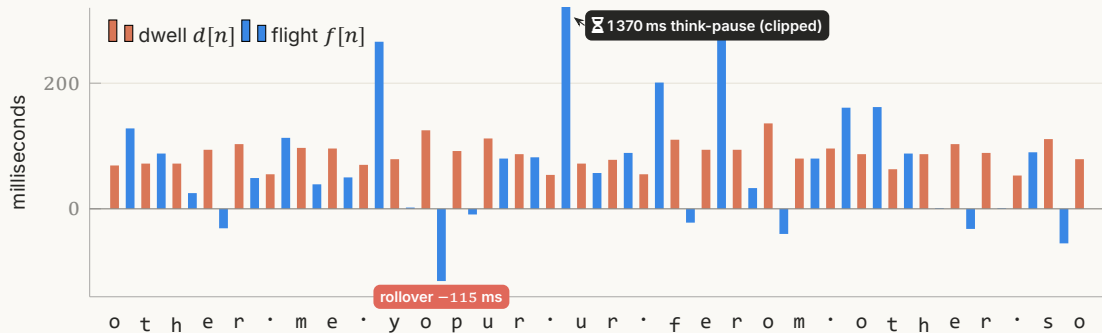
Windows of 32

Enough keys for a stable median / IQR, yet 4–5 samples per test for the fused verdict. A tail under **16 keys** (**MIN_KEYS**) describes noise more than the typist, so it is dropped.

Rebasing

Each window's times are shifted so its first keydown is $t=0$. Only differences are ever used, so no feature changes; every sample just reads like any other.

A real sample, as the pipeline sees it



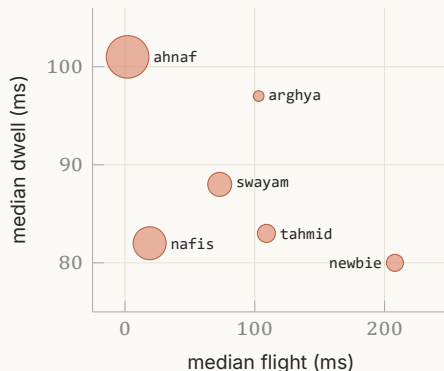
Typed: "other me yopur ur ferom other so", typos and all. The features never read the text, only the timing; free text is the point. One pause is $\approx 20\times$ a normal flight: anything built on a mean or a total would be ruled by it.

What our six real typists look like

user	samples	keys	$\tilde{d}$ ms	$\tilde{f}$ ms	rollover
ahnaf	7	219	101	2	35.8 %
arghya	8	232	97	103	0.0 %
nafis	7	224	82	19	25.3 %
newbie	3	88	80	208	7.1 %
swayam	16	492	88	73	15.3 %
tahmid	7	214	83	109	8.2 %

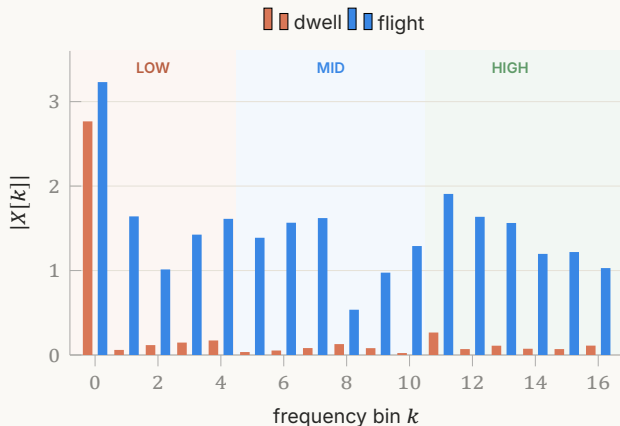
total 48 1469

$\tilde{d}, \tilde{f}$ = median dwell / flight over all of a user's keystrokes.
All captured through the typing test.



bubble size = rollover rate. ahnaf and nafis roll over constantly; arghya never does.

Step 4a · The frequency-domain view of rhythm



The transform

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

$N = 32$. First bring x to that length:
zero-pad, or **resample** by linear interpolation for free text.

5 numbers per stream

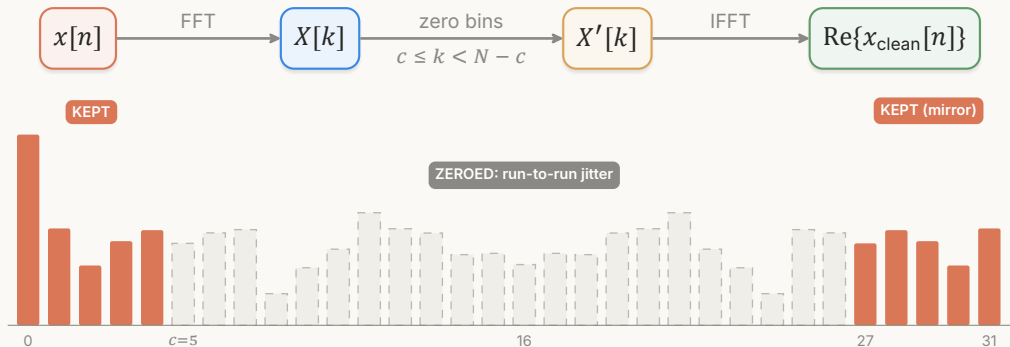
$$\text{centroid} = \sum_k k |X_k| / \sum_k |X_k|$$

$$E_{\text{band}} = \sum_{\text{band}} |X_k|^2 / \sum_k |X_k|^2 \quad (\text{low, mid, high})$$

+ total energy; with mean/std of d, f :
 the original **14-D** vector.

Same real sample. Input is real, so 17 of the 32 bins are unique.

Step 4b · Denoising with an FFT low-pass filter



$c = \max(1, \lfloor \frac{N}{2} \cdot 0.35 \rfloor) = 5$: keep the lowest 35 % of the half-spectrum, zeroing *both* sides so conjugate symmetry holds and the inverse FFT stays real. High frequencies $\approx$ jitter, low frequencies $\approx$ the stable rhythm.

Step 4c · Why the spectrum lost

✂ 1 Too few points

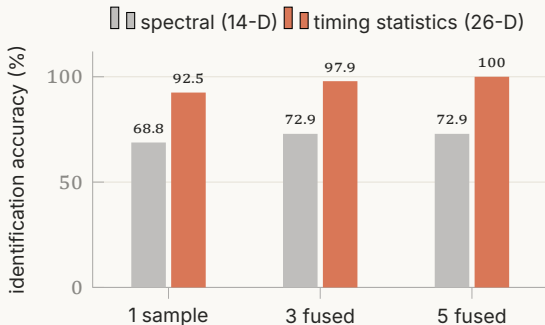
20–40 keys per sample: 17 bins from ~30 noisy points. Three band ratios keep the noise, lose the structure.

📏 2 Length leaks in

Zero-padding makes the spectrum depend on *how long the text was*: fatal on free text.

⌚ 3 One pause rules

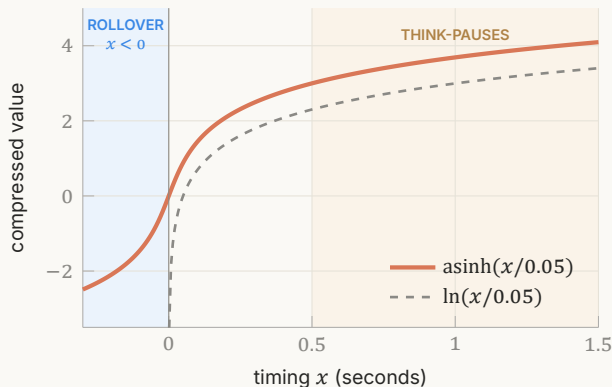
A 1.3 s gap $\approx 20\times$ a 70 ms one: total energy is decided by one hesitation.



Verification EER: **18.72 %** → **5.89 %**

Cross-phrase benchmark. Spectral features added back on top (raw, compressed, 2/4/6 bins) cost accuracy every time, so the spectrum stays for *denoise* and *explain*, not the decision.

Step 5a · Taming pauses and rollover with asinh



$$a(x) = \text{asinh}\left(\frac{x}{s}\right), \quad s = 0.05 \text{ s}$$

- **Linear near zero:** ordinary flights ($\sim 50 \text{ ms} = s$) keep their resolution.
- **Logarithmic in the tails:** a 1.3 s pause is squashed like a log.
- **Sign-aware:** rollover keeps its sign. In can't take $x \leq 0$; clamping would send every rollover to one huge negative value.
- Ratios as $a(x) - a(y)$: a log-ratio normally, still finite as $y \rightarrow 0$.

Step 5b · The 26-dimensional feature vector



Robust before sharp

Medians, IQRs and median-absolute deviations instead of means and standard deviations: one pause must not move the whole vector.

Sign-aware

Flight times go negative. Nothing may assume a positive timing, hence asinh, never log.

Speed-invariant

You type faster fresh, slower tired. Ratios and shape carry identity; absolute speed lives in a few dimensions only.

NaN / $\pm\infty$ from a degenerate sample becomes 0; labels come from **FEATURE_NAMES**.

Block 1 · Robust statistics of each timing stream

code	feature	definition	what it captures
aMu	compressed mean	$\text{mean}(a(x))$	speed, in a symmetric domain
aSd	compressed spread	$\text{std}(a(x))$	scale-free variability
med	median	Q_{50}	typical value
iqr	interquartile range	$Q_{75} - Q_{25}$	spread immune to one pause
mad	median abs. deviation	$\text{median} x - Q_{50} $	robust spread
p90	10–90 span	$Q_{90} - Q_{10}$	width of the bulk
rcv	robust variability	$\text{MAD}/(Q_{50} + 10^{-9})$	robust coefficient of variation
out1	long-outlier share	$\text{Pr}[x > Q_{50} + 2 \text{ MAD}]$	how often they hesitate
roll	rollover share	$\text{Pr}[x < 0]$	flight only: key-overlap habit

For $d[n]$ (8) and $f[n]$ (9). roll is always 0 for dwell (a key can't be released before it's pressed), so dwell omits it.

Blocks 2 & 3 · Key habits and cross-stream ratios

Block 2: key-class habits

code	feature	definition
spDw	space hold	$\tilde{d}_{\text{space}}$
ltDw	letter hold	$\tilde{d}_{\text{letter}}$
s:l	space vs letter	$a(\text{spDw}) - a(\text{ltDw})$
aSp	flight after space	$\tilde{f}$ where key n is space
inW	flight within word	$\tilde{f}$ where key n is not
a:i	word start vs in-word	$a(\text{aSp}) - a(\text{inW})$

Why only the spacebar?

30 keys touch each letter only once or twice, but space is frequent: do you *punch* or *brush* it, and pause between words?

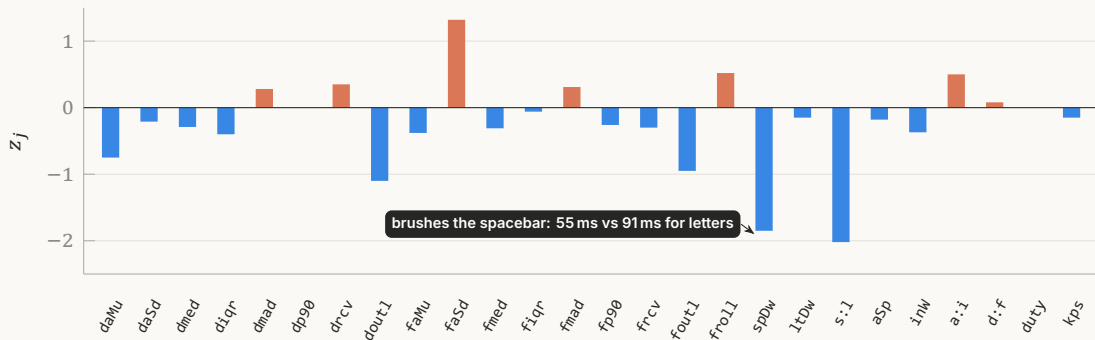
Block 3: cross-stream ratios

code	definition
d:f	$a(\tilde{d}) - a(\tilde{f})$
duty	$\frac{\tilde{d}}{\tilde{d} + \tilde{f} + \varepsilon} \in [0, 1]$
kps	$\frac{N}{\sum d + \sum f }$

Bounded by construction

Tired or fresh, absolute timings move together; *relationships* barely move. A naive $\tilde{d}/\tilde{f}$ explodes for a fast roller (one real sample: $\tilde{f} = 17 \text{ ms} \Rightarrow$ a 5× outlier); these forms stay finite.

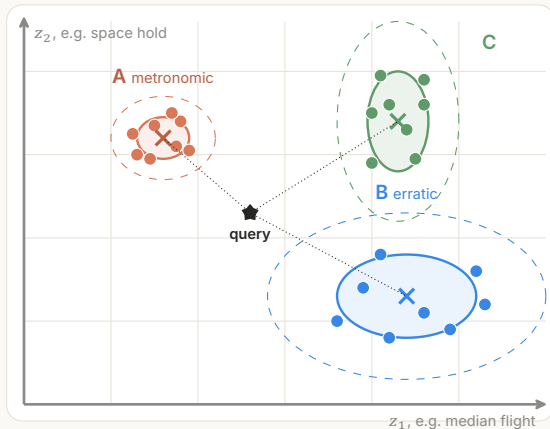
Step 6 · Normalise: one real vector as z-scores



$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \sigma_j \leftarrow \max(\sigma_j, 10^{-8})$$

Feature scales differ wildly (ratios ~0–1, timings ~0.02–0.3 s, kps ~2–8), so every distance works in z-space. The **ZScoreScaler** is **fit on the training / enrolled data only**. In leave-one-out evaluation it is re-fit for every fold, so no test statistic leaks in. ■ above the population mean, ■ below.

Step 7 · Templates: a point *plus* a spread



Template(user_id, mean, std, n)

$$\mu_u = \text{mean}\{z_i : y_i = u\}$$

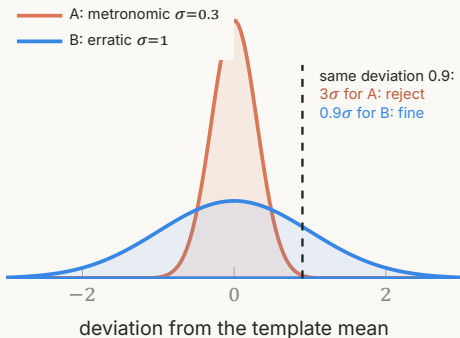
$$\sigma_u = \max(\text{std}\{z_i : y_i = u\}, 0.25 \sigma_{\text{pop}} + 10^{-6})$$

per feature, in z-space, by **build_templates**.

🛡 The spread floor

Enrolled from two samples, a user can show near-zero spread on a feature *purely by luck*; dividing by it would let one coincidence outweigh everything. So spread never drops below 25 % of the population's (**SPREAD_FLOOR**).

Step 8 · Distance: scaled Manhattan



$$D(q, u) = \frac{1}{26} \sum_{j=1}^{26} \frac{|q_j - \mu_{u,j}|}{\sigma_{u,j}}$$

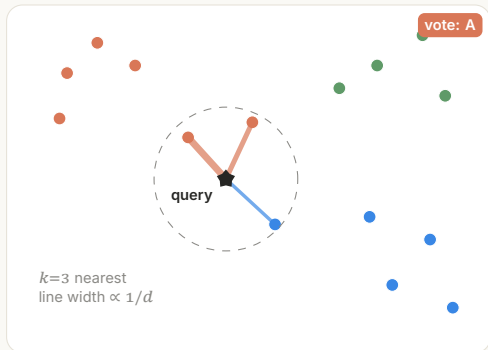
Divide by *that user's spread*

Rejects a metronomic typist for a slip that is normal for an erratic one; a population z-score can't. **EER 7.98 % → 5.45 %**: the biggest single win in the decision layer.

Manhattan, not Euclidean

Squaring lets one wildly-off feature dominate; absolute deviations let the other 25 outvote it.

Step 9 · Identification: distance-weighted k -NN



The vote

$d_i = \sum_j |q_j - x_{ij}|$ sample to sample, in z-space

$$\text{score}(u) = \sum_{i \in \mathcal{N}_3(q), y_i=u} \frac{1}{d_i + 10^{-9}}$$

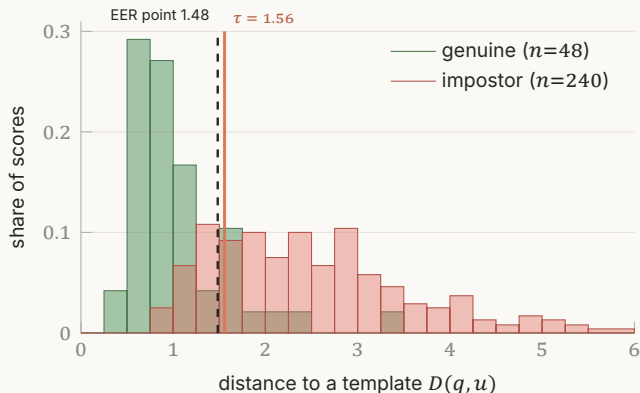
$$\hat{y} = \arg \max_u \text{score}(u)$$

A near-identical neighbour outweighs one that just scraped into the top k , and there is no arbitrary tie-break when k splits evenly.

Leave-one-out cross-validation

For each of the 48 real samples: refit the scaler on the other 47, predict it. $38 / 48 = 79.2\%$ across 6 typists (chance $\approx 17\%$).

Step 10 · Verification: genuine vs impostor



Protocol (**verification_scores**)

For every sample i :

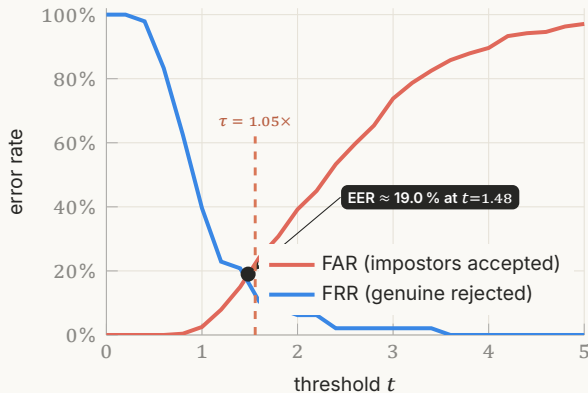
1. refit the scaler *without* i
2. build all templates from the rest
3. **genuine**: $D(x_i, \text{own template})$
4. **impostor**: $D(x_i, \text{every other})$

Genuine mean **1.04** (sd 0.56)

Impostor mean **2.48** (sd 1.06)

Real captures, 6 users. The overlap of the two humps is the error we trade.

Step 11 · Equal error rate and operating point



$$\text{FAR}(t) = \Pr[D_{\text{imp}} \leq t],$$

$\text{FRR}(t) = \Pr[D_{\text{gen}} > t]$. Sweep 500 thresholds;
the EER is where $|\text{FAR} - \text{FRR}|$ is smallest.

τ / t_{EER}	rejected	accepted	HTER
$\times 1.00$	5.8 %	6.1 %	5.95 %
$\times 1.05$	5.0 %	7.3 %	6.13 %
$\times 1.20$ old	2.5 %	11.7 %	7.11 %
$\times 1.30$	0.8 %	14.5 %	7.68 %

cross-phrase benchmark, 8 users. $\text{HTER} = (\text{FAR} + \text{FRR}) / 2$.

ACCEPT_TOLERANCE = 1.05: the fewest total mistakes (exact optimum 1.06 $\times$). On real data it also beats 1.2: 17.3 % vs 19.0 % HTER. τ decides *whether* to answer, never *who*.

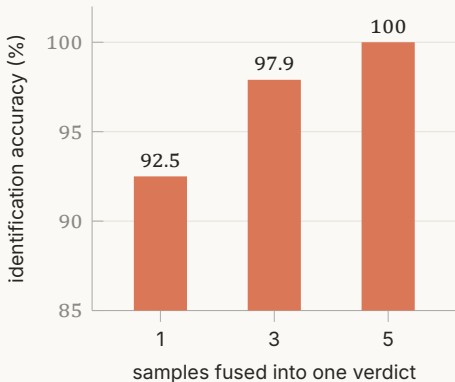
Step 12 · Fusing a whole run into one verdict

sample	A	B	C	per-sample vote
s_1	0.80	1.90	1.40	A
s_2 🔔	1.70	1.50	2.20	B ✖
s_3	0.90	2.10	1.60	A
s_4	1.00	1.80	1.30	A
s_5	0.70	2.00	1.50	A
mean	1.02	1.86	1.60	⇒ A, margin 0.58

illustrative numbers; s_2 : a phone buzzed mid-sentence.

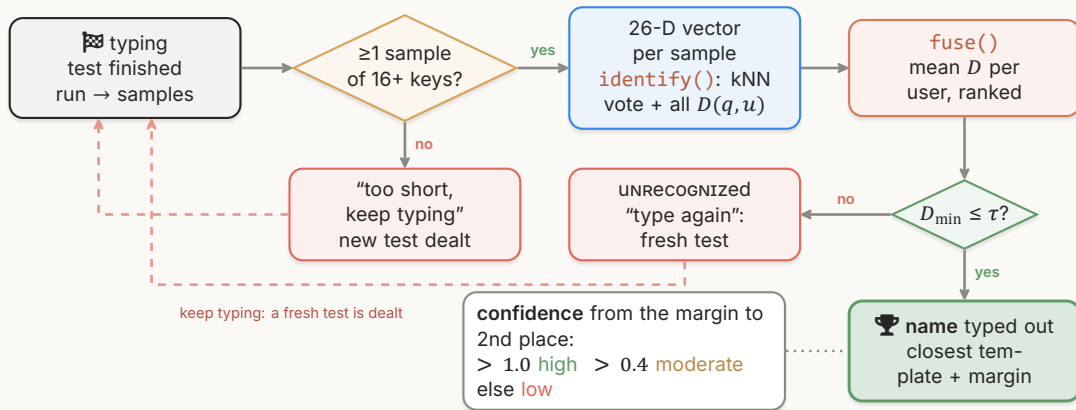
`fuse()`: pool distances, not votes

One run's samples are *known* to share a typist. Averaging distances keeps a second-best sample's evidence; a vote throws it away. Mean beat median and min pooling.



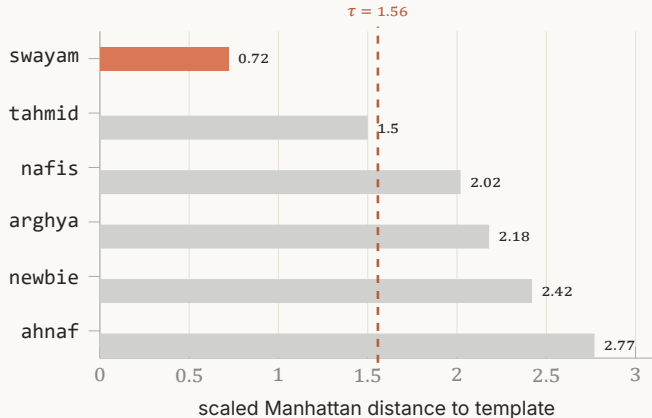
cross-phrase benchmark, leave-one-phrase-out

The decision, end to end



Heavy work (enrolled matrix, EER sweep, identification) runs on a worker thread; the UI polls the result queue every 120 ms and never blocks.

A real identification, bar by bar



VERDICT
swayam

closest template at 0.72 (threshold 1.56)

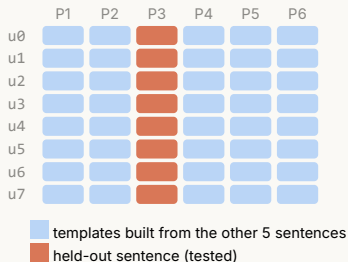
margin over 2nd 0.78 (moderate)

One of swayam's enrolled samples scored against all 6 templates. It is in-sample, so shown only to illustrate the mechanics.

The runner-up, tahmid, sits 0.78 further away: above 0.4 but under 1.0, so the verdict is reported with **moderate confidence**.

How we measured it

Leave-one-*phrase*-out



Leave-one-*sample*-out would compare a sample with siblings from the same sentence, flattering features that encode the sentence, not the typist.

The fixture `tools/benchmark_inference.py`

8 simulated users $\times$ 6 pangrams $\times$ 5 repeats = **240 samples**;
each user has a fixed profile:

parameter	distribution
dwll (mean / sd)	$\mathcal{U}(70, 160) / \mathcal{U}(10, 30)$ ms
flight (mean / sd)	$\mathcal{U}(80, 300) / \mathcal{U}(20, 60)$ ms
per-key bias	$\times \mathcal{U}(0.8, 1.3)$, fixed per key
rollover	$p \sim \mathcal{U}(.03, .22)$; gap = $-\mathcal{U}(.005, .6) \times \text{hold}$
think-pause	$p \sim \mathcal{U}(.01, .06)$; gap +Exp($\mathcal{U}(.15, .6)$ s)

Why add keyboard realism?

The plain generator clamps flights at +10 ms, so it never produces rollover (12.8 % in the fixture, 16.0 % in real captures).

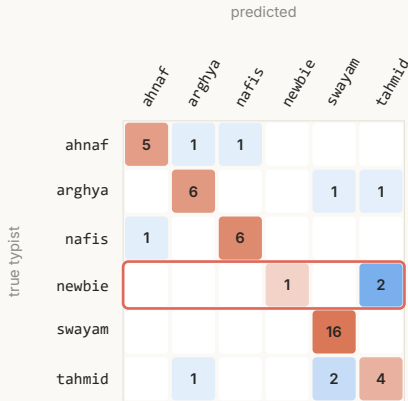
Results

set	vector	dim	1 smp	fuse 3	fuse 5	EER
fixture	spectral	14	68.8	72.9	72.9	18.72
fixture	timing	26	92.5	97.9	100.0	5.89
real	spectral	14	56.2	n/a	n/a	29.79
real	timing	26	79.2	n/a	n/a	18.96

All in %. Fixture: 240 samples, 8 users, leave-one-phrase-out. Real: 48 samples, 6 users, one session each, so only leave-one-sample-out is possible and runs are too short to fuse.

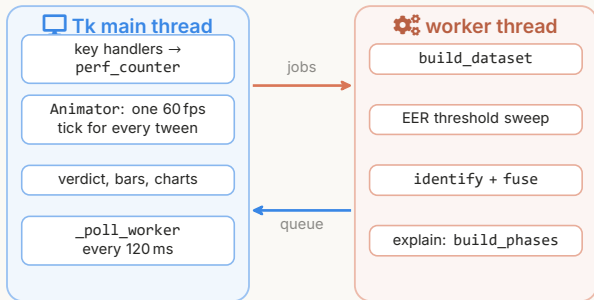
Reading it

The timing vector wins everywhere. Real data is harder: one session per typist, and newbie enrolled only 3 samples.



Real captures, LOOCV k -NN: 38 / 48. Row: newbie, only 3 samples to learn from.

Engineering the app



💡 The splash is not just a curtain

A worker imports numpy + the pipeline and builds the enrolled matrix and EER threshold, so identify is warm on open.

🕒 Motion vs timing fidelity

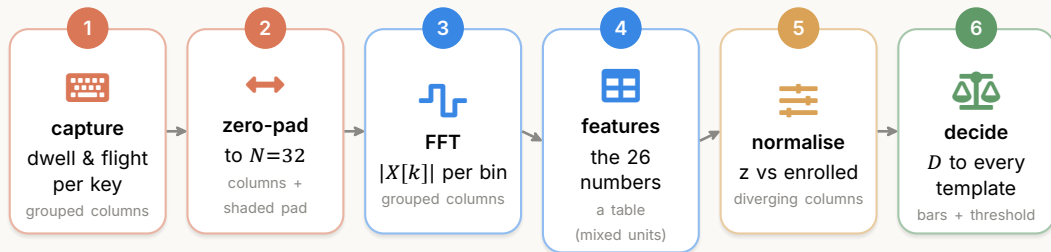
Animations share the thread that timestamps keys. Measured on a synthetic 40 / 60 ms cadence:

	motion off	motion on
flight sd	0.25 ms	0.3 ms
dwel jitter	baseline	+ ≈ 3 ms
dwel worst	baseline	≤ 16 ms
handler bias	≈ 5 ms	≈ 5 ms

Well inside human dwell variability (~ 10 – 30 ms).
Cleanest enrollment:

```
KEYLOGGD_NO_ANIM=1 python main.py
```

Explain mode: the pipeline, one phase at a time



✓ Nothing re-implemented

Each phase calls the *same* functions the identify path calls and just keeps the intermediate values. If the pipeline changes, the explainer changes with it instead of drifting into a pretty lie.

▶ Built to be presented

A phase rail, the chart, and a values panel; step with next / previous or autoplay at 3.2 s per phase. Charts use a CVD-checked terracotta / blue pair on the dark panel.

Running it

```
pip install -r requirements.txt # numpy>=2.4.2

python main.py # splash, then app
python main.py --no-splash
python main.py --data-dir DIR # other enrolled set
KEYLOGGD_NO_ANIM=1 python main.py # cleanest timing

python -m keyloggd.ui.typing_test
python -m keyloggd.pipeline.classifier # LOOCV + EER
python -m keyloggd.pipeline.identify_sample X.json
python -m tools.synthetic_data_generator
python -m tools.benchmark_inference [--real-only]
```

- **Python 3 + tkinter + numpy**, nothing else. kNN, scaler, templates and the EER sweep are written from first principles.
- **identify_sample**: **--k**, **--threshold**, **--tolerance**, **--no-threshold**.
- The benchmark re-checks every accuracy claim in the docstrings.

Takeaways & what's next

Takeaways

- ✓ Typing rhythm identifies people: **92.5 %** from one 32-key sample, **100 %** with five, on the benchmark.
- ✓ **Robust statistics** beat the spectrum for the decision; the spectrum still *explains* rhythm best.
- ✓ **Per-user spread** and **fusion** are the biggest wins of the decision layer.
- ✓ A **game** gets natural typing and many samples, painlessly.

What's next

📅 **More people, more days:** cross-session data to measure drift

🔄 **Adaptive templates:** update slowly as someone's rhythm drifts

👁️ **Continuous mode:** a background check after login

🔒 **Protect the data:** encrypted templates, never raw logs

👤 **Liveness & mimicry tests:** replay and trained-imitator attacks

Thank you

Questions? Or better: come and type.

who is typi|ng right now

